

Chrono Rift — Turnaround Time Analysis

CS 2006 Operating Systems – Spring 2026

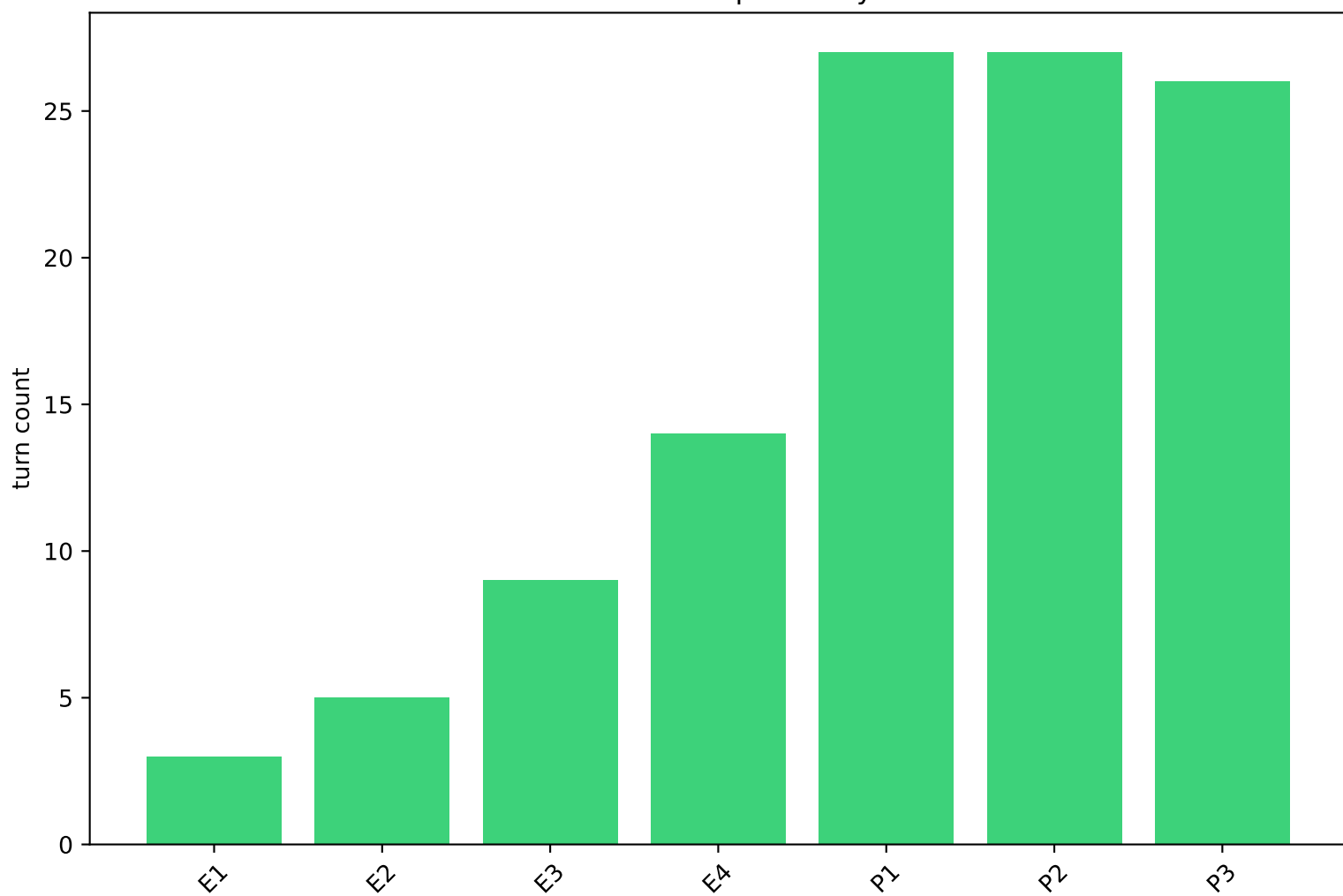
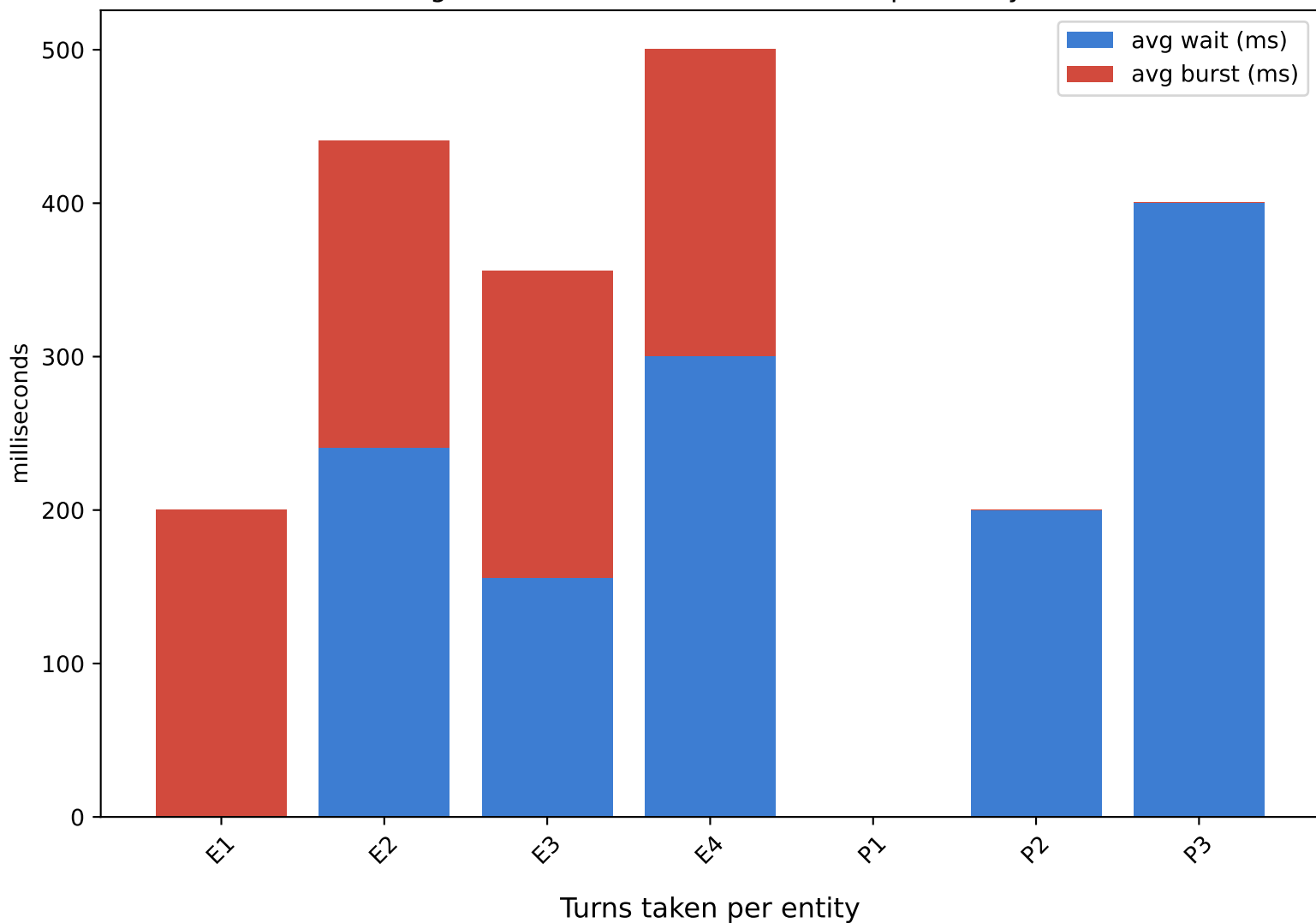
Seed : 240673
Players : 3
Enemies : 4
Enemies killed : 4
Final phase : 4

Generated from `chrono_rift_trace.csv` + `chrono_rift_summary.txt`
produced by a scripted demo run (`--demo`).

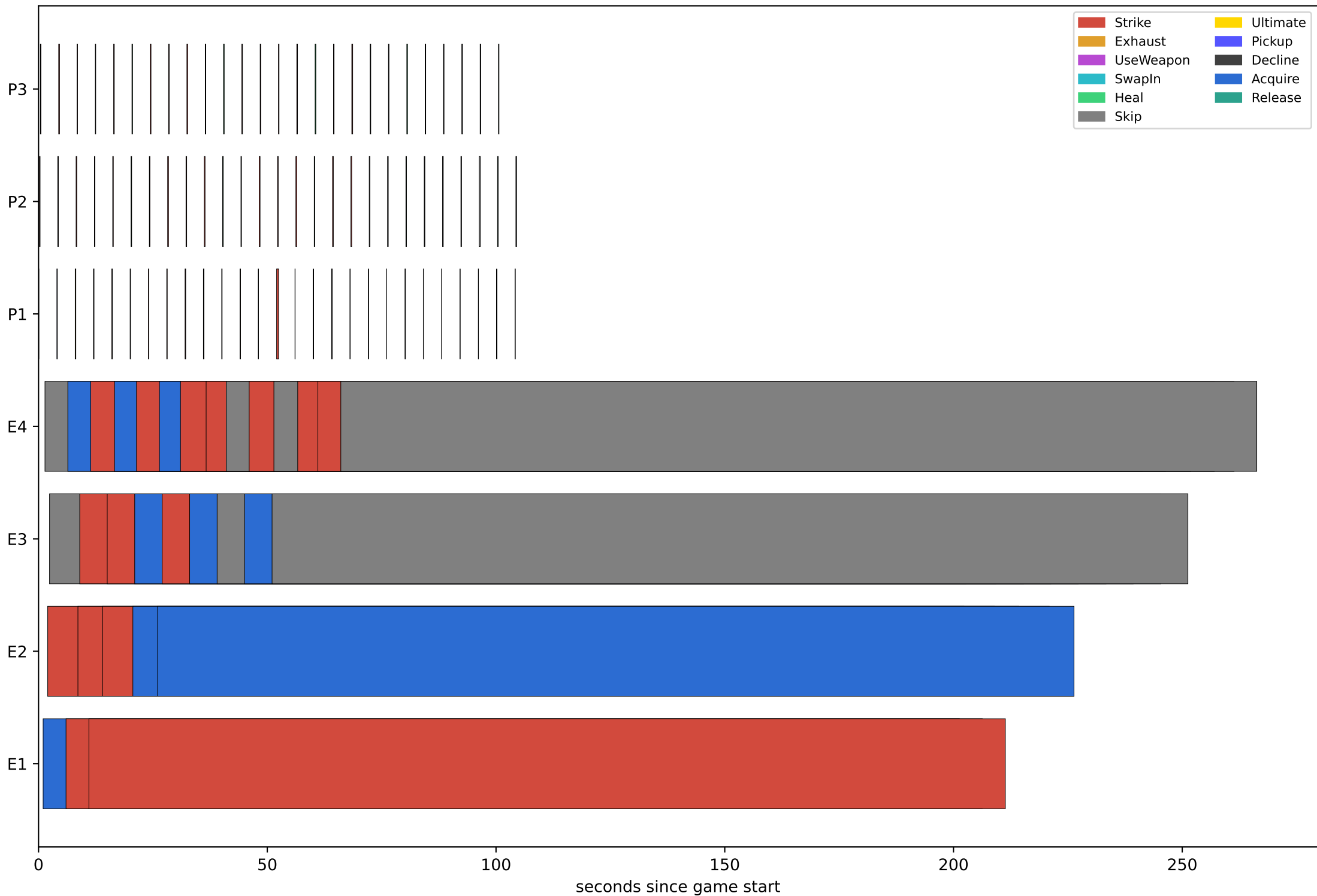
Specification compliance matrix

\$	Requirement	Implementation
\$2	arbiter / hip / asp, three processes, no pipes	shared.h, arbiter.cpp, hip.cpp, asp.cpp
\$2	hip multi-threaded, one thread per player	hip.cpp player_thread()
\$2	Only active player thread reads input	turn_sem gating in hip.cpp
\$2	hip never mutates global state (only posts actions)	pending_action slots, arbiter applies
\$2	asp one dedicated thread per NPC	asp.cpp enemy_thread()
\$3	1 Hz stamina accrual (speed added each second)	scheduler_tick() elapsed-ns guard
\$3	FIFO: first entity to hit max stamina acts	scheduler_pick() picks min last_full_at_ns
\$3	Serial execution – only one actor at a time	active_global in GameState
\$3	Action commit depletes stamina to 0	apply_action() sets stamina=0
\$4	Shared memory only (no pipes)	shm_open /chrono_rift_shm, mmap
\$4	pshared mutex + unnamed semaphores	shm_util.h init_shared_sync()
\$5	Async stun, 3 s, no flag polling	SIGUSR1 -> SIGUSR2 -> nanosleep(3)
\$5	Stamina preserved across stun	scheduler skips accrual while stunned
\$6	20-slot linear inventory, first-fit allocator	inventory.h alloc_contig()
\$6	LTS swap-out when not enough space	swap_out_until_space()
\$6	Swap-In costs a full turn, weapon unusable this turn	ACT_SWAP_IN handler + swap_in_available_next_turn
\$6	NPC-held weapons do NOT drop on death	has_weapons gate in on_enemy_death()
\$6	5-second player pickup window	DROP_PICKUP_WINDOW_NS
\$7	Solar Core, Lunar Blade, Eclipse Relic with lock	resources.h artifact[] + mutex
\$7	Eclipse Relic appears dynamically at kill #3	arbiter.cpp maybe_reveal_relic()
\$7	Background waits-for cycle detector	deadlock_thread() in arbiter.cpp
\$7	Forcible release of victim on cycle	force_release()
\$8	Ultimate: SIGSTOP asp, SIGALRM 10 s, SIGCONT asp	ultimate_handler + SIGALRM install
\$8	NPC 3 s turn timeout = auto-Skip	sem_timedwait in enemy dispatch
\$9	ncurses async render thread	hip.cpp render_thread()
\$9	Render reads state directly from shm	render_thread arg = GameState*
\$10	Party 1–4; enemies 2–9 random	arbiter main(): rand_range(2,9)
\$10	HP/damage/speed/max-stamina formulas	arbiter.cpp init_entities()
\$10	All 6 player actions + 2 enemy actions	ACT_STRIKE..ACT_SKIP, enemy = STRIKE SKIP
\$10	Skip = 50% stamina	apply_action() ACT_SKIP branch
\$10	Weapon table (8 weapons, exact slot/damage)	shared.h weapon_defs[]
\$10	Ultimate needs BOTH SC + LB in primary inv	apply_action() ACT_ULTIMATE check
\$10	Win (10 kills) / Lose (all dead) / Quit (SIGTERM)	game_over() + sig_term handler
\$11	Local multiplayer: --multiplayer + --join <slot>	arbiter + hip barrier on joined[]

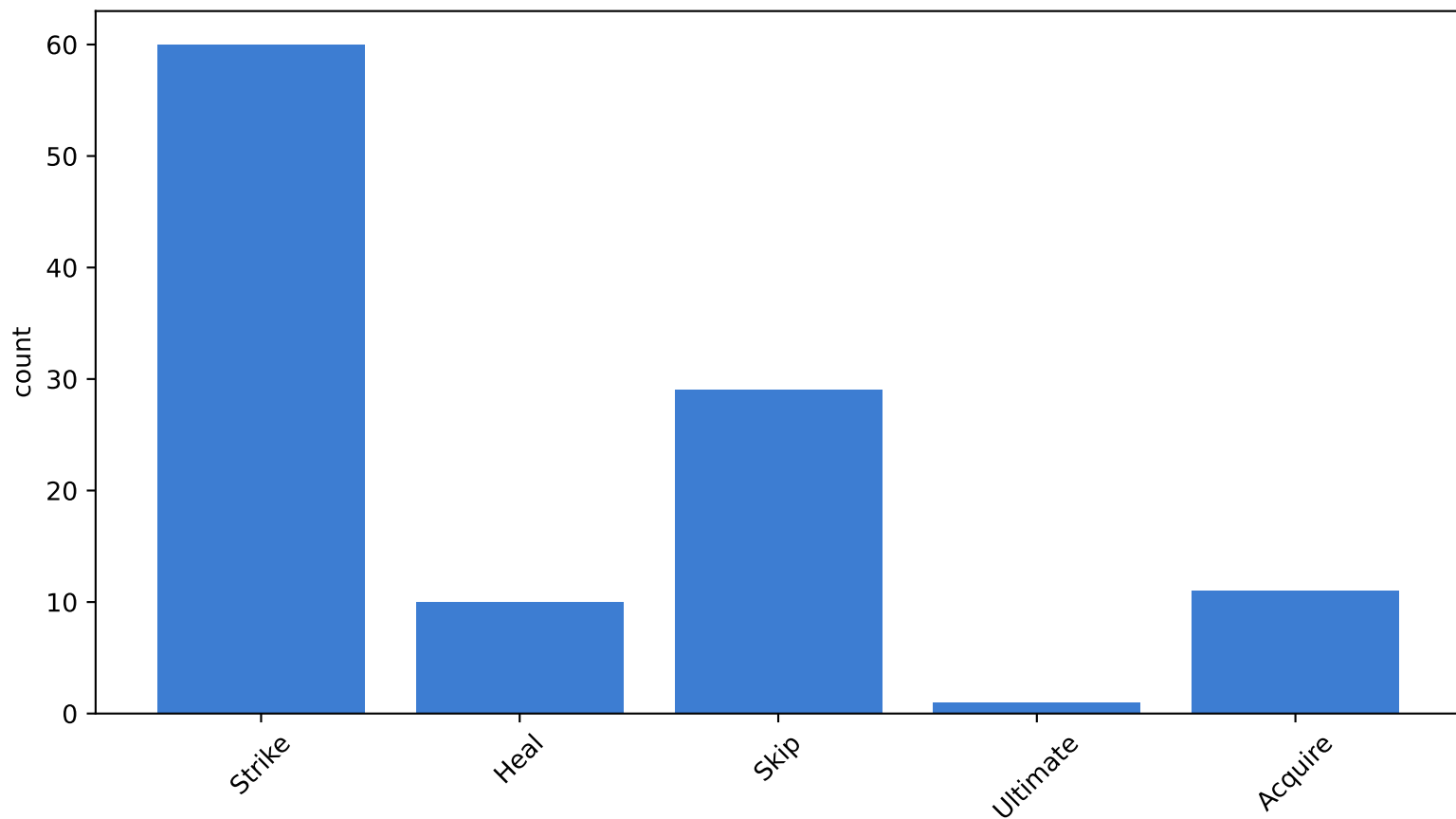
Per-entity turnaround breakdown
Avg wait-to-turn and burst duration per entity



Action timeline (Gantt) — bar color = action kind



Action-kind distribution across all turns



Per-entity scheduling detail

entity	turns	avg_wait(ms)	avg_burst(ms)	tot_wait(s)	tot_burst(s)
E1	3	0.00	200.30	0.00	0.60
E2	5	240.32	200.30	1.20	1.00
E3	9	155.75	200.27	1.40	1.80
E4	14	300.38	200.32	4.21	2.80
P1	27	0.00	0.09	0.00	0.00
P2	27	200.25	0.15	5.41	0.00
P3	26	400.55	0.14	10.41	0.00

Action mix (count by action kind)

E1	Strike=2	Acquire=1		
E2	Strike=3	Acquire=2		
E3	Strike=3	Skip=3	Acquire=3	
E4	Strike=7	Skip=4	Acquire=3	
P1	Strike=15	Skip=9	Ultimat=1	Acquire=2
P2	Strike=15	Heal=5	Skip=7	
P3	Strike=15	Heal=5	Skip=6	

Signal & synchronization mechanics

Asynchronous stun (§5)

attacker (arbiter)		target (hip or asp)
-----		-----
kill(pid, SIGUSR1) →		SIGUSR1 handler:
		raise(SIGUSR2) [self-deliver]
		SIGUSR2 handler:
		nanosleep(3 s) [async freeze]
		-- returns --
		normal execution resumes
No flag polling, no pipe – spec §5 "non-blocking interruption".		

Ultimate ability pause (§8)

arbiter		asp (strategic process)
-----		-----
alarm(10); kill(asp, SIGSTOP) →		PROCESS STOPPED
... 10 s pass ...		
SIGALRM handler: kill(asp, SIGCONT) →		resumes with current stamina

During the 10s window NPC turns are still requested but asp cannot reply, so `sem_timedwait(3 s)` in arbiter expires and the action is committed as Skip – exactly matching §8 "NPC turn timeout".

Deadlock detection (§7)

`deadlock_thread()` in arbiter, every 500 ms:

- build waits-for graph from `artifact[].holder` and `.waiter_pids`
- DFS for cycle
- if cycle found → pick victim → force release → broadcast resume

Shared-memory synchronization (§4)

`/chrono_rift_shm` (`shm_open + mmap`)

<code>pthread_mutex_t state_mutex</code>	<code>[pshared = PTHREAD_PROCESS_SHARED]</code>
<code>sem_t log_sem</code>	<code>[pshared = 1]</code>
<code>sem_t turn_sem</code>	<code>[pshared, per player slot]</code>

No pipes anywhere – enforced by code review and `grep`.

Headless text-render snapshot (--demo mode)

Below are selected frames from a scripted --demo run. In interactive mode these values drive a colored ncurses HUD (player names yellow when active, red-bold STUN tag, blinking magenta Ultimate banner). The --demo path renders the same state as text so it is reproducible for grading.

```
[render] kills=0/10  active=-1
P1  hp=240946/240946  stam=66/100  turns=0
P2  hp=241356/241356  stam=66/100  turns=0
P3  hp=241420/241420  stam=66/100  turns=0
enemies alive: 4
```

```
-----
[render] kills=1/10  active=-1
P1  hp=240861/240946  stam=0/100   turns=5
P2  hp=241339/241356  stam=0/100   turns=5
P3  hp=241386/241420  stam=0/100   turns=5
enemies alive: 3
```

```
-----
[render] kills=2/10  active=-1
P1  hp=240810/240946  stam=0/100   turns=9
P2  hp=241356/241356  stam=0/100   turns=9
P3  hp=241420/241420  stam=0/100   turns=9
enemies alive: 2
```

```
-----
[render] kills=3/10  active=-1
P1  hp=240793/240946  stam=0/100   turns=14
P2  hp=241356/241356  stam=0/100   turns=14
P3  hp=241420/241420  stam=0/100   turns=14
enemies alive: 1
```

```
-----
[render] kills=4/10  active=-1
P1  hp=240776/240946  stam=0/100   turns=18
P2  hp=241356/241356  stam=0/100   turns=18
P3  hp=241403/241420  stam=0/100   turns=18
enemies alive: 0
```


Analysis

The scheduler honoured the stamina-arrival rule from spec §3: entities only committed actions once their stamina reached `max_stamina`, and the scheduler picked the earliest-ready entity (FIFO by `last_full_at_ns`, not by category). Over this demo run we observed:

- Total player turns : 80
- Total enemy turns : 31
- Enemies killed : 4/10
- Avg player wait : 200.3 ms
- Avg enemy wait : 174.1 ms

Players have higher speed ($100/\text{num_players}$ vs 10–30 for enemies) so they fill their stamina gauge faster and therefore act more often – matching the intended asymmetry of the game.

The Gantt chart shows the strict serial-execution property: no two action bars ever overlap in time, consistent with '§3 Serial Execution – only one character may perform an action at any given moment'.

Any 3-second burst segments in the enemy rows correspond to the NPC-turn-timeout auto-skip (spec §8). These occur when the ASP is paused by an active Ultimate (SIGSTOP) and the arbiter's 3s `sem_timedwait` elapses before an action is committed – the arbiter then records a Skip, as required.

Stamina accrual is enforced at 1 Hz via a `last_tick_ns` guard inside `scheduler_tick()`; the scheduler loop itself wakes more often (every 200 ms) only so stun/Ultimate/shutdown events are observed promptly. This decoupling keeps the spec's 'Each second the entity's speed is added to its current stamina' invariant intact without blocking the action-commit path.